COMPUTER SCIENCE

PROJECT ON
RESTAURANT SYSTEM

Academic Year: 2025-26

Project submitted in partial fulfilment of Practical Examination for Class XII, All India Senior School Certificate Examination

SUBMITTED TO:

Mrs. Nithiya.R

SUBMITTED BY:

Name: Deveshh Suresh

School ID No.:2023

Class: XII- A1

SUGUNA PIP SCHOOL, COIMBATORE.

(CBSE, Affiliation No. 1930213)

SUGUNA PIP SCHOOL, COIMBATORE

CERTIFICATE

COMPUTER SCIENCE PROJECT

This is to certify that the Computer Science project report titled RESTAURANT SYSTEM which has been submitted to SUGUNA PIP School, meeting the requirements of the CBSE Board practical examination for the academic year 2025 – 2026, is a bonafide work done by DEVESHH SURESH.

Signature of Teacher Incharge | Signature of Principal

Signature of Internal Examiner | Signature of External Examiner

ACKNOWLEDGEMENT

I would like to thank **Mr. POOVANNAN**, Principal, Suguna PIP school, Coimbatore for his immeasurable direction towards the course of action and support throughout the project.

I would also like to thank my Faculty Guide, **Mrs. NITHIYA.R** for her valuable mentoring throughout the project. The interesting lectures of my knowledgeable teacher have helped me understand the various concepts related to python and MySQL.
I got ample opportunity to do research, which enriched and broadened my knowledge and understanding of this area.

Thanks to my parents, friends, and everyone else who have been directly or indirectly supportive during my project. I am seriously indebted to them.

DECLARATION

I, Deveshh Suresh, student of Class XII-A1, Suguna PIP School, Coimbatore (Affiliation No. 1930213), located at Nehru Nagar, Kalapatti Road, Coimbatore, Tamil Nadu-641014, hereby declare that the project titled “Restaurant Dining and Staff System” submitted for Internal Assessment in Class XII is my original work. This project has been completed under the supervision of Mrs. Nithiya.R, Faculty of Computer Science Department, Suguna PIP School, Coimbatore. I certify that the contents of this report are the result of my own efforts and have not been copied from any other source.

I, hereby certify that the work contained in this report is original and has been completed by me under the supervision of the faculty.

Student Name : Deveshh Suresh

School ID No. : 2023

Register No. :

Class : 12A1

ABSTRACT

This project presents the Lexmi Nicos Order Suite, a sophisticated, multi-tiered digital ordering system developed using Python's Tkinter library for the Graphical User Interface (GUI) and MySQL for secure data persistence.

The application's central elegance lies in its stateful and modular architecture, which seamlessly guides users through a complex selection process. The system begins with a critical authentication layer, differentiating access between staff and guests, and providing specific verification for members against a connected MySQL database. The core functionality involves a dynamically linked ordering process across Appetizers, Entrees, and Desserts. Crucially, the system employs a synchronized network of global variables that ensures the master order list is updated in real-time across all navigation screens upon submission. This persistent data model culminates in a Final Order Summary, which not only presents a definitive review of the customer's selections but also incorporates an intuitive error-correction loop, allowing instant navigation back to any category for modification. The Lexmi Nicos Order Suite stands as a demonstration of effective, user-centric application design, unifying a sleek front-end experience with a robust, database-driven back-end logic.

CONTENTS

INTRODUCTION

Welcome to the Lexmi Nicos Order Suite, a system architected for operational precision, dedicated to elevating the efficiency and reliability of the front-of-house ordering experience. This suite provides a streamlined, rigorously authenticated pathway from client identification to final order compilation. The project's central aim is to deliver a robust, error-proof solution that eliminates transactional ambiguity and synchronizes real-time data flow across all user interfaces.

This sophisticated application is constructed upon a core foundation of Python and key external modules, each fulfilling a vital function in the system's integrity:

Module	Primary Function
tkinter	Graphical User Interface (GUI): Utilized to construct the multi-window environment, including the dynamic root, authentication screens (wind5, wind6), main menu (wind3), and category interfaces.
mysql.connector	Database Integration: Essential for establishing and managing the connection to the lexminicos database, enabling secure Member and Staff Authentication via credential lookup.

tk.StringVar	State Management & Data Capture: Employed locally within category windows (e.g., Appetizers) to temporarily bind and capture precise quantity inputs from the entry fields before the final submission and global update.

SYSTEM FEATURES AND OPERATIONAL FUNCTIONALITIES

The Lexmi Nicos Order Suite is defined by its structured workflow and critical functional components:

- Multi-Tiered Authentication: The system provides distinct access control for Staff (via staff() function) and Guests (via guest() function), further segmenting guests into Members (requiring passcode verification) and non-members for tailored service.
- Persistent Order State: Order data is managed using global list variables (appsorder, entreesorder, dessertsorder). This ensures that selections are continuously tracked and never lost when navigating between the Appetizer, Entree, and Dessert windows.
- Real-Time Synchronization: The Submit command in each category window triggers a critical synchronization function, which updates the global lists and instantaneously rebuilds the master order list, displaying the complete order live within the current category screen.
- Intuitive Error Correction Loop: The finaliseorder() window presents the complete, final selection summary while retaining all category navigation buttons. This design allows users to spot an error, jump back to the specific category, correct the quantity, resubmit, and return to the final summary with changes immediately reflected.

- Dynamic Menu Presentation: Item lists are dynamically bound to entry fields, allowing users to input exact quantities for any of the 30 available items across the three primary categories.

MY SQL TABLE DESIGN

```
mysql> use lexminicos;
Database changed
mysql> select*from staff;
+---------+----------+
| name    | password |
+---------+----------+
| DEVESHH | 1        |
| SURESH  | 2        |
+---------+----------+
2 rows in set (0.00 sec)

mysql> select*from members;
+---------+----------+
| name    | password |
+---------+----------+
| DEVESHH | 1        |
| SURESH  | 2        |
+---------+----------+
2 rows in set (0.00 sec)

mysql> desc staff;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| name     | varchar(20) | YES  |     | NULL    |       |
| password | varchar(10) | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
2 rows in set (0.06 sec)
```

```
mysql> desc members;
+----------+-------------+------+-----+---------+-------+
| Field    | Type        | Null | Key | Default | Extra |
+----------+-------------+------+-----+---------+-------+
| name     | varchar(20) | YES  |     | NULL    |       |
| password | varchar(10) | YES  |     | NULL    |       |
+----------+-------------+------+-----+---------+-------+
2 rows in set (0.00 sec)
```

SOURCE CODE

```
import tkinter
import mysql.connector
tk = tkinter

mycon = mysql.connector.connect(host="localhost", user="root",
password="Suresh23", database="lexminicos")
cursor = mycon.cursor()

import mysql.connector
#-----------MYSQL-------------------
mycon = mysql.connector.connect(host="localhost",
user="root",password="Suresh23",database="lexminicos" )
cursor = mycon.cursor()

def fetch_members():
   """Fetches member name and password from the 'members' table and
returns them as a dictionary."""
   cursor.execute("SELECT name, password FROM members")
   member_data = dict(cursor.fetchall())
   return member_data

def fetch_staff():
   """Fetches staff name and password from the 'staff' table and returns them
as a dictionary."""
   cursor.execute("SELECT name, password FROM staff")
   staff_data = dict(cursor.fetchall())
   return staff_data
# ---------- MAIN WINDOW ----------

#BLOCK 1 CRETE THE HOMEEPAGE OR STARTING PAGE,    BLOCK 2
SHOWS MENU     BLOCK 3 ASKS IF MEMBER OR NOT    BLOCK 4 IS IF STAFF
root = tk.Tk()
root.title('MENU')
root.geometry('1500x600')
root.configure(bg='black')

# global variables for selections
```

```
s = ''  # will store final order as string
i = tk.IntVar()  # variable for member/guest selection
i.set(3)  # initial value

final = []  # final combined order list
appsorder = []  # order quantities for appetizers
entreesorder = []  # order quantities for entrees
dessertsorder = []  # order quantities for desserts

# ---------- BLOCK 2 FUNCTION TO SHOW MENU BASED ON
MEMBER/GUEST ----------
def showmenu():
    # if a choice has been made (member or not)
    if i.get() != 3:
        global wind2
        wind2.withdraw()  # hide the guest/member selection window

    # if not a member
    if i.get() == 0:
        discount = 0  # placeholder for discounts if needed
        menu()  # open main menu
    else:
        # member login window
        global wind5
        wind5 = tk.Toplevel(root)
        wind5.title('Member Verification')
        wind5.geometry('600x300')
        wind5.configure(bg='black')

        # Name Label and Entry
        namelabel = tk.Label(wind5, text='Name:', fg='white', bg='black',
font=('Arial', 12))
        namelabel.place(x=50, y=40)
        name = tk.Entry(wind5, width=30, bg='white', font=('Arial', 12))
        name.place(x=150, y=40)

        # Passcode Label and Entry
        passcodelabel = tk.Label(wind5, text='Passcode:', fg='white', bg='black',
font=('Arial', 12))
        passcodelabel.place(x=50, y=90)
```

```
    passcode = tk.Entry(wind5, width=30, bg='white', show='*', font=('Arial',
12))
    passcode.place(x=150, y=90)

    members = fetch_members() # Fetches data from MySQL

    # check login credentials
    def check():
        namegiven = name.get()
        code = passcode.get()
        a=0
        for i in members.keys():
            if namegiven in members.keys():
                if code == members[namegiven] and i==namegiven:
                    a=a+1
        if a==0:
            result = tk.Label(wind5, text='ACCESS DENIED', fg='white',
bg='black', font=('Arial', 12, 'bold'))
            result.place(x=220, y=180)
        else:
            menu()

    # Submit button to verify member login
    submit = tk.Button(wind5, text='SUBMIT', font=('Arial', 12, 'bold'),
                        fg='black', bg='#E16C2E', width=10, height=1,
command=check)
    submit.place(x=250, y=140)

# ---------- BLOCK 3 FUNCTION FOR GUEST SELECTION ----------
def guest():
    global i, wind2
    root.withdraw()  # hide main root window
    wind2 = tk.Toplevel(root)
    wind2.geometry('1500x600')
    wind2.configure(bg='black')

    # Radio button for member
    member = tk.Radiobutton(wind2, text='member', font=(30), fg='black',
bg='#E16C2E', value=1, variable=i)
    member.place(x=50, y=100)
```

```
    # Radio button for non-member
    notmember = tk.Radiobutton(wind2, text='not a member', font=(30),
fg='black', bg='#E16C2E', value=0, variable=i)
    notmember.place(x=300, y=100)

    # Submit button to continue
    submit = tk.Button(wind2, text='SUBMIT', font=(30), fg='black',
bg='#E16C2E', command=showmenu)
    submit.place(x=175, y=300)

    # Instructions button
    how = tkinter.Button(wind2, text='INSTRUCTIONS', font=('Arial', 14),
fg='black', bg='#E16C2E', command=lambda: [wind2.withdraw(), howto()])
    how.place(x=500, y=400)

# ---------- BLOCK 4 FUNCTION FOR STAFF SELECTION ----------
def staff():
    root.withdraw()
    wind6 = tk.Tk()
    wind6.geometry('600x300')
    wind6.title('Staff Verification')
    wind6.configure(bg='black')

    # Name label and entry
    namelabel = tk.Label(wind6, text='Name:', fg='white', bg='black',
font=('Arial', 12))
    namelabel.place(x=50, y=40)
    name = tk.Entry(wind6, width=30, bg='white', font=('Arial', 12))
    name.place(x=150, y=40)

    # Passcode label and entry
    passcodelabel = tk.Label(wind6, text='Passcode:', fg='white', bg='black',
font=('Arial', 12))
    passcodelabel.place(x=50, y=90)
    passcode = tk.Entry(wind6, width=30, bg='white', show='*', font=('Arial',
12))
    passcode.place(x=150, y=90)

    employee = fetch_staff() # uses my sql
```

```
    # check login credentials
    def check():
        namegiven = str(name.get())
        code =str(passcode.get())
        a=0
        for i in employee.keys():
            if i==namegiven and code == employee[i]:
#if namegiven not even in employee defenitely the 2nd condition will not be
checked so then no error
                a+=1
        if a==0:
            result = tk.Label(wind6, text='ACCESS DENIED', fg='white', bg='black',
font=('Arial', 12, 'bold'))
            result.place(x=220, y=180)
        else:
            result = tk.Label(wind6, text='ACCESS GRANTED', fg='white',
bg='black', font=('Arial', 12, 'bold'))
            result.place(x=220, y=180)

    # Submit button to verify staff login
    submit = tk.Button(wind6, text='SUBMIT', font=('Arial', 12, 'bold'),
fg='black', bg='#E16C2E', width=10, height=1, command=check)
    submit.place(x=250, y=140)

# ---------- STAFF AND GUEST BUTTONS IN MAIN WINDOW FROM BLOCK
1 ----------
staff_btn = tk.Button(root, text='click if staff', font=(30), fg='black',
bg='#E16C2E', command=staff)
staff_btn.place(x=50, y=100)

guest_btn = tk.Button(root, text='click if guest', font=(30), fg='black',
bg='#E16C2E', command=guest)
guest_btn.place(x=200, y=100)

# ---------- FOOD LISTS ----------
appslist = [
    "Spinach Artichoke Dip", "Crispy Spring Rolls", "Caprese Skewers", "Mini
Crab Cakes",
```

```
    "Garlic Parmesan Breadsticks", "Loaded Potato Skins", "Shrimp Cocktail",
    "Hummus Platter", "Chicken Satay", "Bruschetta"
]

entreeslist = [
    "Grilled Salmon", "Chicken Tikka Masala", "Classic Beef Lasagna",
"Mushroom Risotto",
    "Pan-Seared Duck Breast", "Spicy Shrimp Scampi", "Vegetable Korma",
    "New York Strip Steak", "Lamb Shank", "Black Bean Burger"
]

dessertslist = [
    "Chocolate Lava Cake", "New York Style Cheesecake", "Tiramisu", "Apple
Crumble",
    "Crème Brûlée", "Brownie Sundae", "Key Lime Pie", "Red Velvet Cake",
    "Sticky Toffee Pudding", "Mango Mousse"
]

# ---------- INSTRUCTIONS WINDOW ----------
def howto():
    global howwind
    howwind1 = tk.Toplevel(root)
    howwind1.geometry('1500x900')
    howwind1.configure(bg='black')
    howwind1.title('How to Use the App')

    instructions = (
        "📝 HOW TO USE:\n\n"
        "• If you're a member, select 'Member' and enter any text for Name and
Passcode, then click 'Login'.\n"
        "• If you're not a member, choose 'Not a Member' and click 'SUBMIT' to
open the main 'LEXMI NICOS' menu.\n\n"
        "• On the menu screen, click 'Appetizers', 'Entrees', or 'Desserts' to
browse food categories.\n"
        "• In each category, type the quantity you want next to each item (leave
it blank or 0 if you don’t want that item).\n"
        "• Important: click 'Submit' at the top-left to save your selections before
moving to another category.\n\n"
        "• Use the 'Go to' buttons to navigate between categories.\n"
        "• Click 'Back to Menu' anytime to return to the main menu.\n"
```

```
        "• Once done, click 'Finalise Order' to review your complete order.\n"
        "• To make changes, go back using the 'Go to' buttons, edit quantities,
click 'Submit' again, and re-finalize your order.\n\n"
        "• To exit, simply close the main 'MENU' window using the 'X'
button.\n\n"
        "💡 Tips:\n"
        "  - Always click 'Submit' after entering quantities.\n"
        "  - Use only whole numbers (no decimals or letters).\n"
        "  - This version does not calculate prices."
    )

    h = tk.Label(
        howwind1,
        text=instructions,
        font=('Arial', 14),
        fg='#E16C2E',
        bg='black',
        justify='left',
        wraplength=1450
    )
    h.pack(padx=30, pady=30, anchor='w')

    back = tk.Button(
        howwind1,
        text='BACK',
        font=('Arial', 14, 'bold'),
        width=25,
        bg='#E16C2E',
        fg='black',
        command=lambda: [howwind1.destroy(), wind2.deiconify()]
    )
    back.pack(pady=20)

# ---------- MENU WINDOW ----------
def menu():
    global wind3
    wind3 = tkinter.Toplevel(root)
    wind3.title('MENU')
    wind3.geometry('1500x600')
    wind3.configure(bg='black')
```

```
    wlcm = tkinter.Label(wind3, text="WELCOME TO", font=('TIMES NEW
ROMAN', 12), fg='white', bg='black')
    wlcm.place(x=740, y=0)

    ttl = tkinter.Label(wind3, text="LEXMI NICOS", font=('ELEPHANT', 40),
fg='#E16C2E', bg='black')
    ttl.place(x=570, y=20)

    # Category buttons
    appsbutton = tkinter.Button(wind3, text="Appetizers", font=('Arial', 14),
fg='black', bg='#E16C2E', width=15, height=2, command=apps)
    appsbutton.place(x=50, y=120)

    entreesbutton = tkinter.Button(wind3, text="Entrees", font=('Arial', 14),
fg='black', bg='#E16C2E', width=15, height=2, command=entrees)
    entreesbutton.place(x=250, y=120)

    dessertsbutton = tkinter.Button(wind3, text="Desserts", font=('Arial', 14),
fg='black', bg='#E16C2E', width=15, height=2, command=desserts)
    dessertsbutton.place(x=450, y=120)

# ---------- FINAL ORDER WINDOW ----------
def finaliseorder():
    global finalwind
    finalwind = tkinter.Toplevel(root)
    finalwind.title('FINAL ORDER')
    finalwind.geometry('1500x900')
    finalwind.configure(bg='black')

    title_label = tk.Label(finalwind, text="YOUR ORDER", font=('ELEPHANT',
20), fg='#E16C2E', bg='black')
    title_label.place(x=600, y=20)

    # Buttons to go back to categories or main menu
    apps_button = tk.Button(finalwind, text="Go to Appetizers", font=('Arial',
12), fg='black', bg='#E16C2E', command=lambda: [finalwind.withdraw(),
apps()])
    apps_button.place(x=150, y=20 * (len(dessertslist) + 6))
```

```
    entrees_button = tk.Button(finalwind, text="Go to Entrees", font=('Arial',
12), fg='black', bg='#E16C2E', command=lambda: [finalwind.withdraw(),
entrees()])
    entrees_button.place(x=300, y=20 * (len(dessertslist) + 6))

    desserts_button = tk.Button(finalwind, text="Go to Desserts", font=('Arial',
12), fg='black', bg='#E16C2E', command=lambda: [finalwind.withdraw(),
desserts()])
    desserts_button.place(x=450, y=20 * (len(dessertslist) + 6))

    back_button = tk.Button(finalwind, text="Back to Menu", font=('Arial', 12),
fg='black', bg='#E16C2E', command=lambda: [finalwind.withdraw(), menu()])
    back_button.place(x=600, y=20 * (len(dessertslist) + 6))

    # prepare final order list
    global final
    final = []
    for i in range(len(appslist)):
        if i < len(appsorder) and appsorder[i] != "0":
            final.append(appslist[i] + ", " + appsorder[i])

    for i in range(len(entreeslist)):
        if i < len(entreesorder) and entreesorder[i] != "0":
            final.append(entreeslist[i] + ", " + entreesorder[i])

    for i in range(len(dessertslist)):
        if i < len(dessertsorder) and dessertsorder[i] != "0":
            final.append(dessertslist[i] + ", " + dessertsorder[i])

    # display final order
    for i in final:
        j = tk.Label(finalwind, text=i, font=(10), fg='white', bg='black')
        j.place(x=0, y=60 + 25 * (final.index(i)))

# ---------- APPETIZERS WINDOW ----------
def apps():
    global finalwind
    wind3.withdraw()
    appswind = tkinter.Toplevel(root)
    appswind.title('APPETIZERS')
```

```
    appswind.geometry('1500x900')
    appswind.configure(bg='black')
    global tapps
    tapps = []
#-----------FINDING THE DISH WITH THE BIGGEST
NAME-------------------------------
    e = ''
    for i in appslist:
        if len(i) > len(e):
            e = i
#-----------------------------------------------------------------------
    # create labels and entry fields for each appetizer
    for i in appslist:
        entry = tk.StringVar()
        entry.set('0')

        j = tk.Label(appswind, text=i, font=('Arial', 12), fg='white', bg='black')
        j.place(x=0, y=20 * appslist.index(i))

        k = tk.Entry(appswind, textvariable=entry, width=5, font=('Arial', 12))
        k.place(x=7*len(e)+100, y=20 * appslist.index(i))
        #THE ENTRY BOXES TO BE PLACED PERFECTLY USE THE LENGTH OF
THE LONGEST DISHES NAME ALDREADY CALCULATED

        tapps.append(entry)

        '''NOTE: here it is like a list of lists, to explain it,
conside a list L=[a,b,c,d] here the letters correspond to the value stored in
the entries and tapps is L. now when the user changes the entry  in tapps,
the value a is changed, so that way when submit is hit we can et the values'''

    # function to save selections
    def appsfunc():
        global appsorder, final, entreesorder, dessertsorder
        appsorder = []
        for i in tapps:
            appsorder.append(i.get())

        final = []
        for i in range(len(appslist)):
```

```
      #since the tapps has entries stored as strings i am checking which
and all in apps order have non 0 entries and put themalong with dish name
into final

      if i < len(appsorder) and appsorder[i] != "0":
        final.append(appslist[i] + ", " + appsorder[i])

    for i in range(len(entreeslist)):
      if i < len(entreesorder) and entreesorder[i] != "0":
        final.append(entreeslist[i] + ", " + entreesorder[i])

    for i in range(len(dessertslist)):
      if i < len(dessertsorder) and dessertsorder[i] != "0":
        final.append(dessertslist[i] + ", " + dessertsorder[i])

    # display current selections
    global s
    s = ''
    for i in final:
      s = s + '\n' + i
    ordertext = tk.Label(appswind, text=s, bg='black', fg='white', font=(12))
    ordertext.place(x=800, y=0)

  Submit = tk.Button(appswind, text='Submit', font=('Arial', 14), fg='black',
bg='#E16C2E', command=appsfunc)
  Submit.place(x=0, y=20*(len(appslist) + 3))

  # buttons to navigate categories
  def open_entrees():
    appswind.withdraw()
    entrees()

  def open_desserts():
    appswind.withdraw()
    desserts()

  entrees_button = tk.Button(appswind, text="Go to Entrees", font=('Arial',
12), fg='black', bg='#E16C2E', command=open_entrees)
  entrees_button.place(x=150, y=20 * (len(appslist) + 3))
```

```
    desserts_button = tk.Button(appswind, text="Go to Desserts", font=('Arial',
12), fg='black', bg='#E16C2E', command=open_desserts)
    desserts_button.place(x=300, y=20 * (len(appslist) + 3))

    back_button = tk.Button(appswind, text="Back to Menu", font=('Arial', 12),
fg='black', bg='#E16C2E', command=lambda: [appswind.withdraw(),
menu()])
    back_button.place(x=450, y=20 * (len(appslist) + 3))

#when researching for the code i discovered that instead of defining a
function for each button i can just use the keyword lambda and define the
command there itself

    Finalise = tk.Button(appswind, text='Finalise order', font=('Arial', 12),
fg='black', bg='#E16C2E', command=lambda: [appswind.withdraw(),
finaliseorder()])
    Finalise.place(x=300, y=20*(len(appslist)+6))

# ---------- ENTREES WINDOW ----------
def entrees():
    wind3.withdraw()
    entreewind = tkinter.Toplevel(root)
    entreewind.title('ENTREES')
    entreewind.geometry('1500x900')
    entreewind.configure(bg='black')

    t = []

    e = ''
    for i in entreeslist:
        if len(i) > len(e):
            e = i

    # create labels and entry fields for each entree
    for i in entreeslist:
        entry = tk.StringVar()
        entry.set('0')

        j = tk.Label(entreewind, text=i, font=('Arial', 12), fg='white', bg='black')
        j.place(x=0, y=20 * entreeslist.index(i))
```

```
        k = tk.Entry(entreewind, textvariable=entry, width=5, font=('Arial', 12))
        k.place(x=7*len(e)+100, y=20 * entreeslist.index(i))

        t.append(entry)

    # function to save selections
    def entreesfunc():
        global entreesorder, appsorder, dessertsorder
        entreesorder = []
        for i in t:
            entreesorder.append(i.get())

        final = []
        for i in range(len(appslist)):
            if i < len(appsorder) and appsorder[i] != "0":
                final.append(appslist[i] + ", " + appsorder[i])

        for i in range(len(entreeslist)):
            if i < len(entreesorder) and entreesorder[i] != "0":
                final.append(entreeslist[i] + ", " + entreesorder[i])

        for i in range(len(dessertslist)):
            if i < len(dessertsorder) and dessertsorder[i] != "0":
                final.append(dessertslist[i] + ", " + dessertsorder[i])

        global s
        s = ''
        for i in final:
            s = s + '\n' + i
        ordertext = tk.Label(entreewind, text=s, bg='black', fg='white', font=(12))
        ordertext.place(x=800, y=0)

    Submit = tk.Button(entreewind, text='Submit', font=('Arial', 14), fg='black',
bg='#E16C2E', command=entreesfunc)
    Submit.place(x=0, y=20*(len(entreeslist) + 3))

    # buttons to navigate categories
    def open_apps():
        entreewind.withdraw()
```

```
        apps()

    def open_desserts():
        entreewind.withdraw()
        desserts()

    apps_button = tk.Button(entreewind, text="Go to Appetizers", font=('Arial',
12), fg='black', bg='#E16C2E', command=open_apps)
    apps_button.place(x=150, y=20 * (len(entreeslist) + 3))

    desserts_button = tk.Button(entreewind, text="Go to Desserts",
font=('Arial', 12), fg='black', bg='#E16C2E', command=open_desserts)
    desserts_button.place(x=300, y=20 * (len(entreeslist) + 3))

    back_button = tk.Button(entreewind, text="Back to Menu", font=('Arial',
12), fg='black', bg='#E16C2E', command=lambda: [entreewind.withdraw(),
menu()])
    back_button.place(x=450, y=20 * (len(entreeslist) + 3))

    Finalise = tk.Button(entreewind, text='Finalise order', font=('Arial', 12),
fg='black', bg='#E16C2E', command=lambda: [entreewind.withdraw(),
finaliseorder()])
    Finalise.place(x=300, y=20*(len(entreeslist)+6))

# ---------- DESSERTS WINDOW ----------
def desserts():
    wind3.withdraw()
    dessertswind = tkinter.Toplevel(root)
    dessertswind.title('DESSERTS')
    dessertswind.geometry('1500x900')
    dessertswind.configure(bg='black')

    t = []

    e = ''
    for i in dessertslist:
        if len(i) > len(e):
            e = i

    # create labels and entry fields for each dessert
```

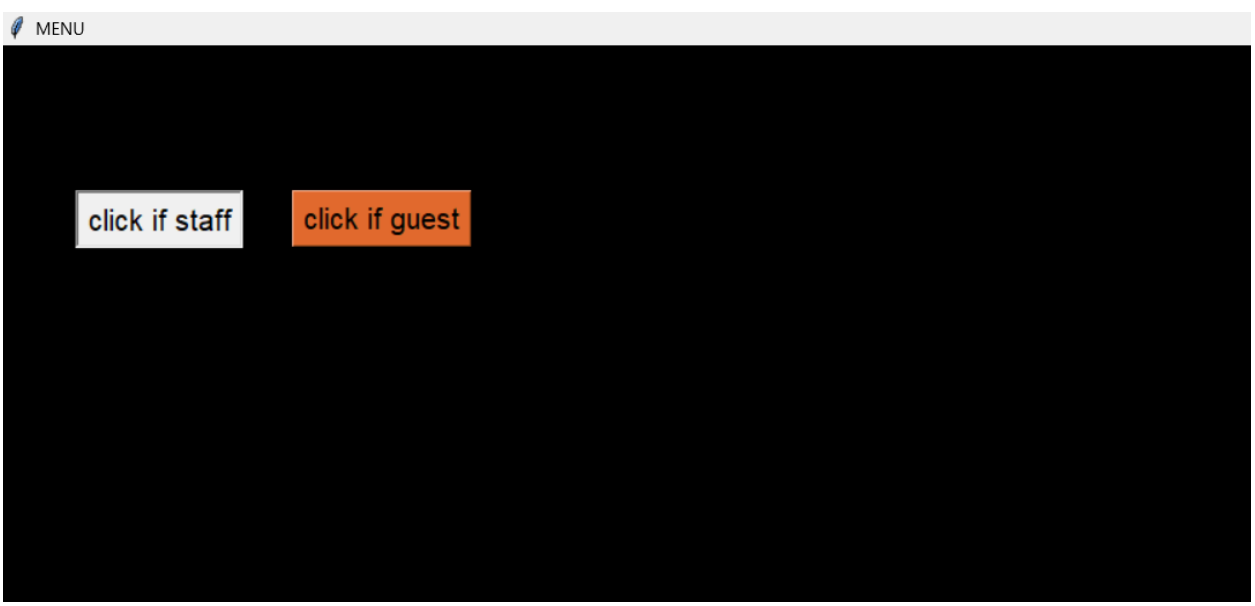
```
    for i in dessertslist:
        entry = tk.StringVar()
        entry.set('0')

        j = tk.Label(dessertswind, text=i, font=('Arial', 12), fg='white', bg='black')
        j.place(x=0, y=20 * dessertslist.index(i))

        k = tk.Entry(dessertswind, textvariable=entry, width=5, font=('Arial', 12))
        k.place(x=7*len(e)+100, y=20 * dessertslist.index(i))

        t.append(entry)

    # function to save selections
    def dessertsfunc():
        global dessertsorder, appsorder, entreesorder
        dessertsorder = []
        for i in t:
            dessertsorder.append(i.get())

        final = []
        for i in range(len(appslist)):
            if i < len(appsorder) and appsorder[i] != "0":
                final.append(appslist[i] + ", " + appsorder[i])

        for i in range(len(entreeslist)):
            if i < len(entreesorder) and entreesorder[i] != "0":
                final.append(entreeslist[i] + ", " + entreesorder[i])

        for i in range(len(dessertslist)):
            if i < len(dessertsorder) and dessertsorder[i] != "0":
                final.append(dessertslist[i] + ", " + dessertsorder[i])

        global s
        s = ''
        for i in final:
            s = s + '\n' + i
        ordertext = tk.Label(dessertswind, text=s, bg='black', fg='white',
font=(12))
        ordertext.place(x=800, y=0)
```

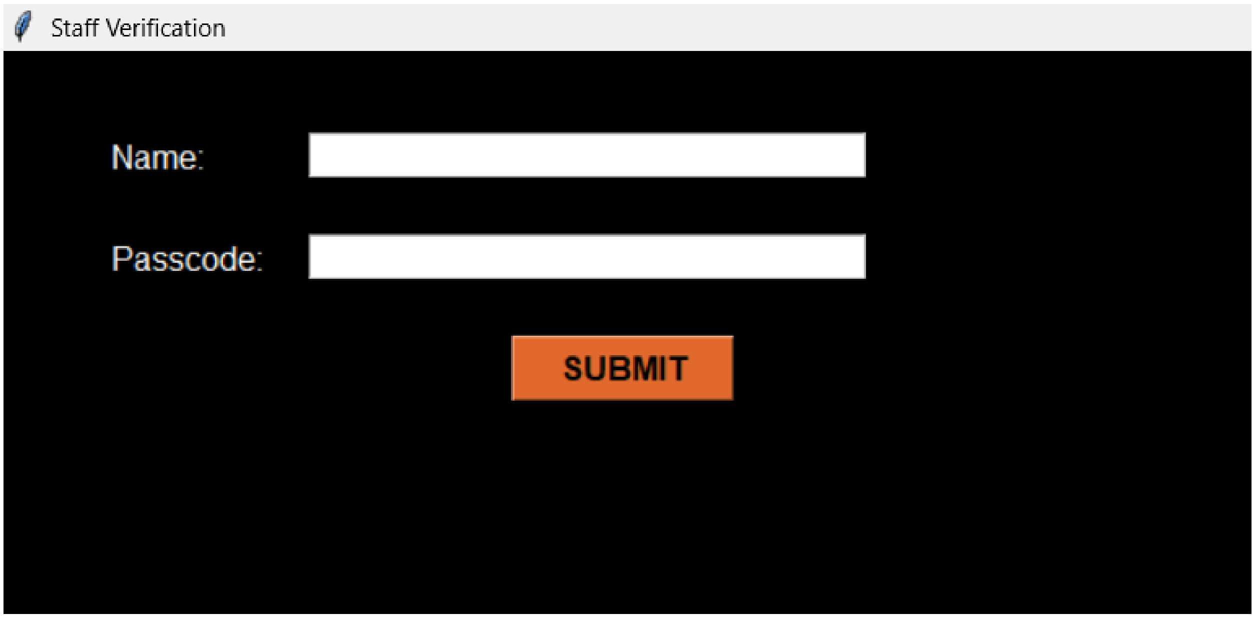
```
    Submit = tk.Button(dessertswind, text='Submit', font=('Arial', 14),
fg='black', bg='#E16C2E', command=dessertsfunc)
    Submit.place(x=0, y=20*(len(dessertslist) + 3))

    # buttons to navigate categories
    def open_apps():
        dessertswind.withdraw()
        apps()

    def open_entrees():
        dessertswind.withdraw()
        entrees()

    apps_button = tk.Button(dessertswind, text="Go to Appetizers",
font=('Arial', 12), fg='black', bg='#E16C2E', command=open_apps)
    apps_button.place(x=150, y=20*(len(dessertslist)+3))

    entrees_button = tk.Button(dessertswind, text="Go to Entrees",
font=('Arial', 12), fg='black', bg='#E16C2E', command=open_entrees)
    entrees_button.place(x=300, y=20*(len(dessertslist)+3))

    back_button = tk.Button(dessertswind, text="Back to Menu", font=('Arial',
12), fg='black', bg='#E16C2E', command=lambda: [dessertswind.withdraw(),
menu()])
    back_button.place(x=450, y=20*(len(dessertslist)+3))

    Finalise = tk.Button(dessertswind, text='Finalise order', font=('Arial', 12),
fg='black', bg='#E16C2E', command=lambda: [dessertswind.withdraw(),
finaliseorder()])
    Finalise.place(x=300, y=20*(len(dessertslist)+6))

# ---------- RUN THE APP ----------
root.mainloop()
mycon.close()
```

OUTPUT

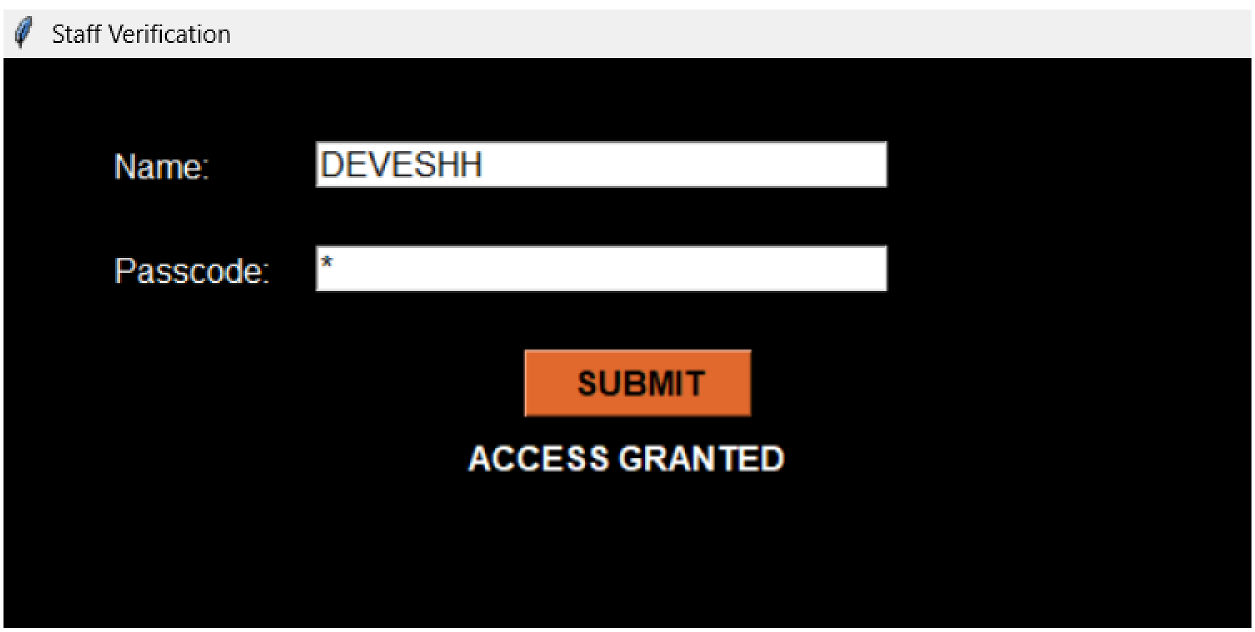
As soon as program starts running

1. Staff uses it(click if staff is clicked)

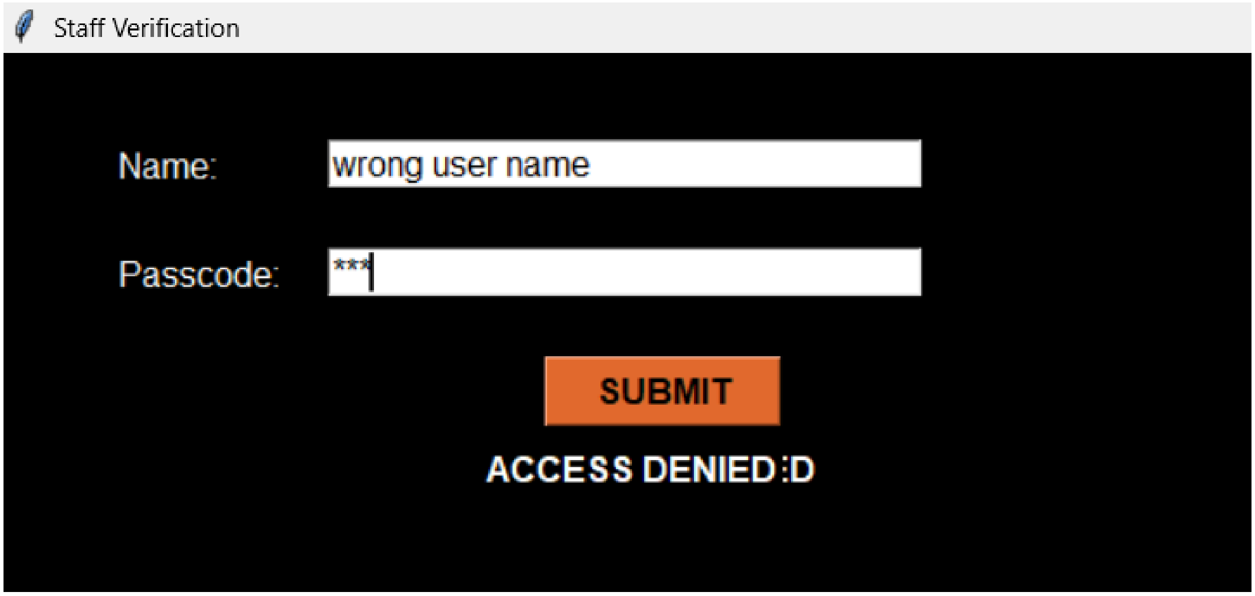




If correct user name and password is selected the below image shows what happens

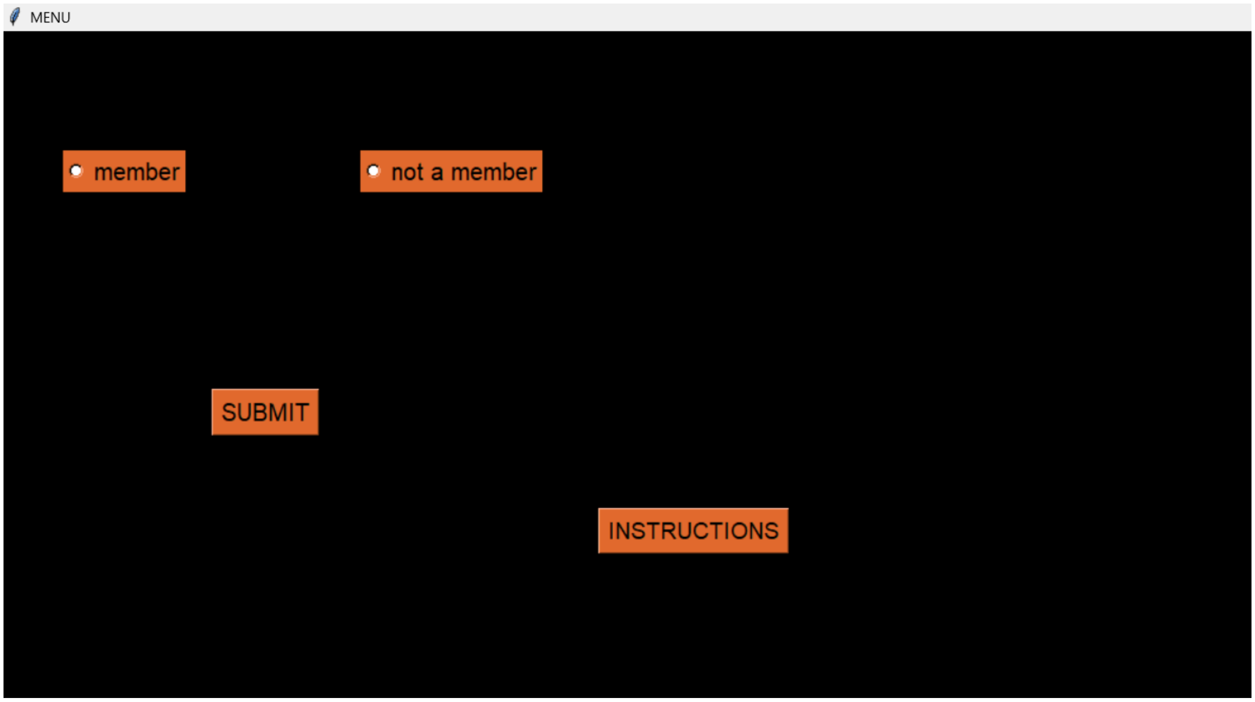


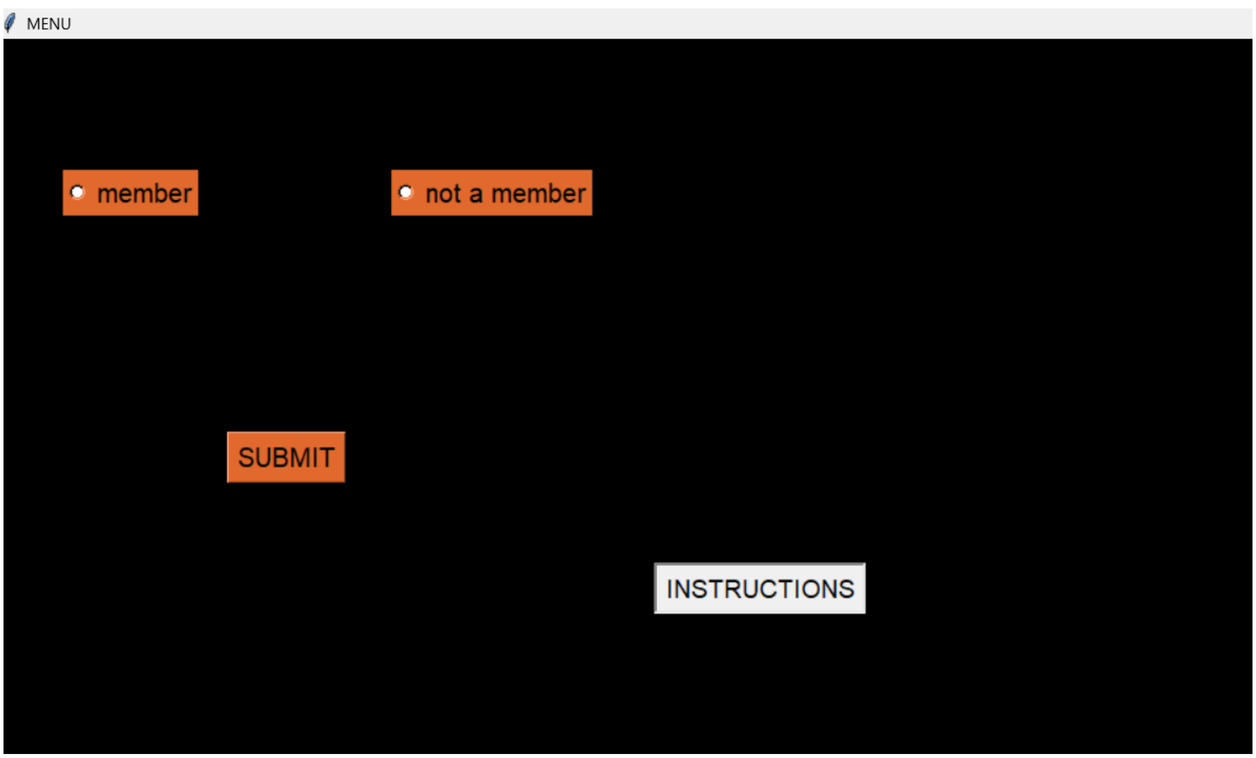
If incorrect username password combination is entered the following image illustrates what happens

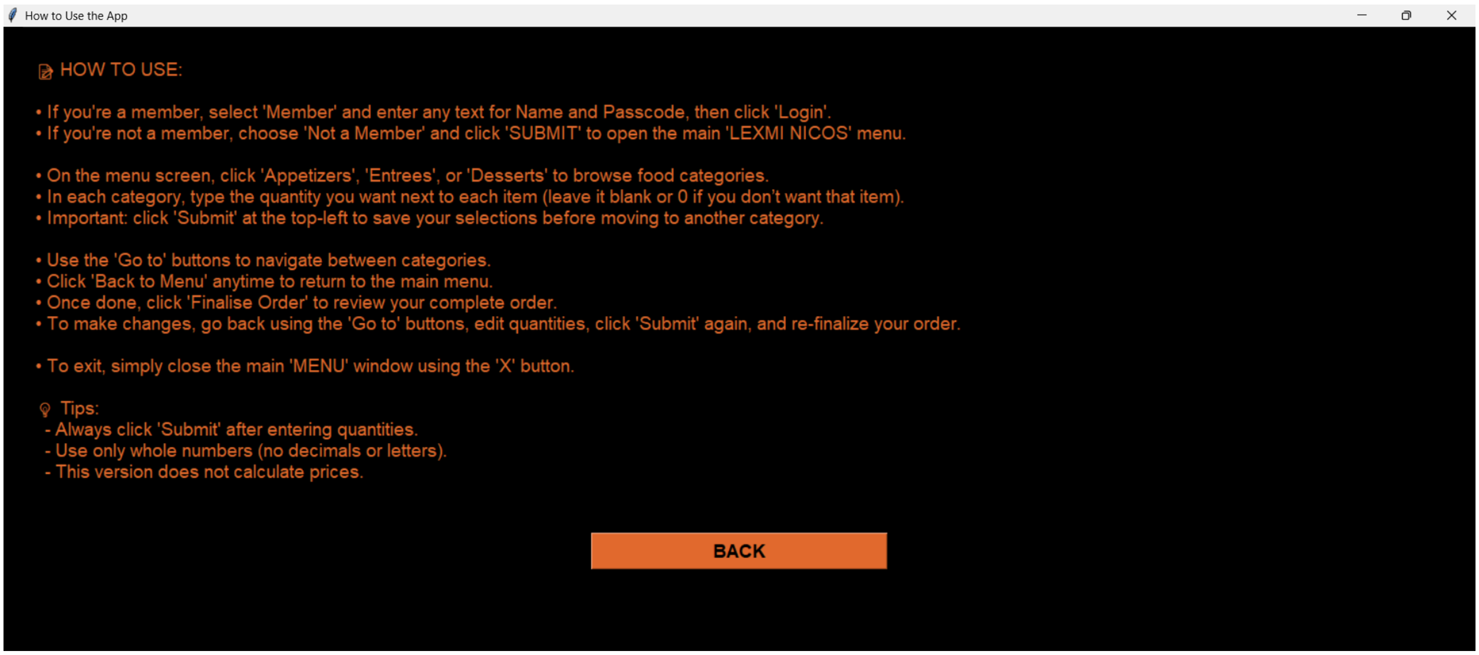

Staff Verification
Name:
wrong user name
Passcode:
***
SUBMIT
ACCESS DENIED:D

2. If guest


MENU
click if staff
click if guest


MENU
member
not a member
SUBMIT
INSTRUCTIONS

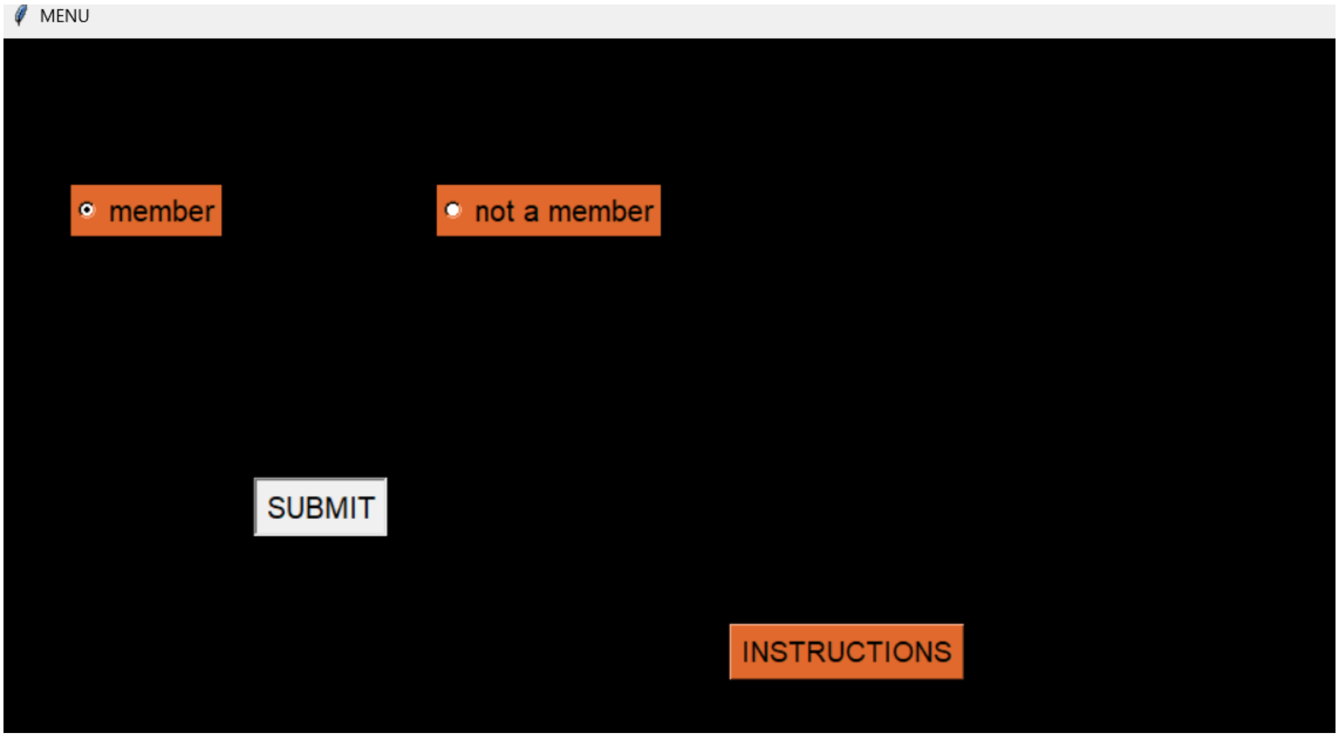

MENU
member
not a member
SUBMIT
INSTRUCTIONS

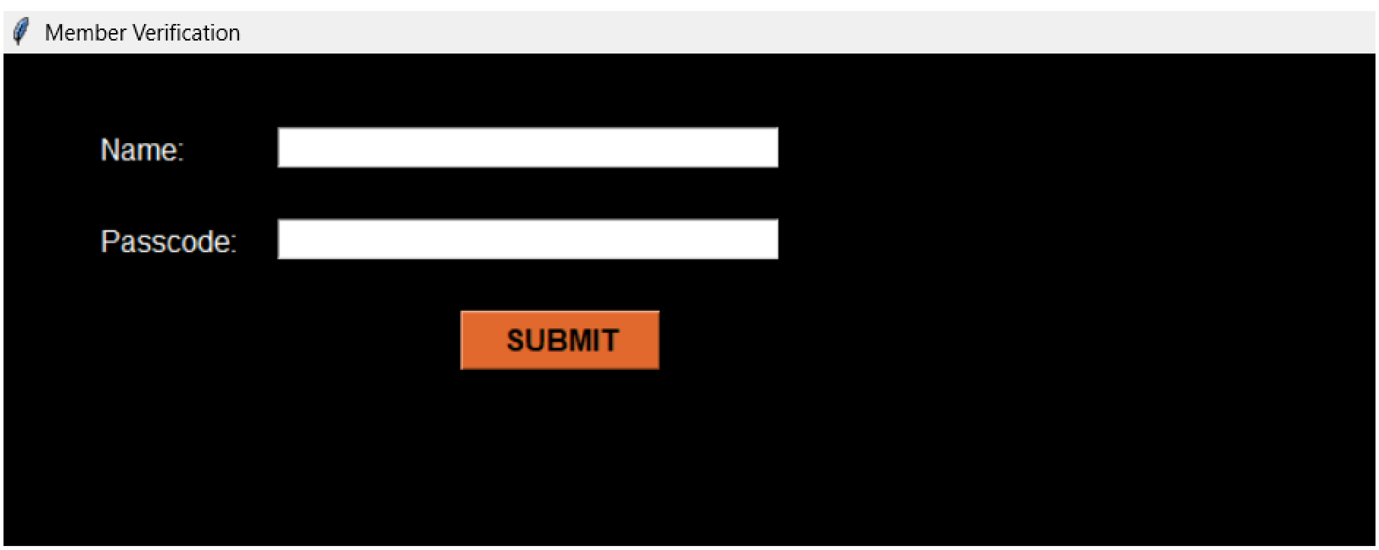


Clicking instructions opens the instructions menu

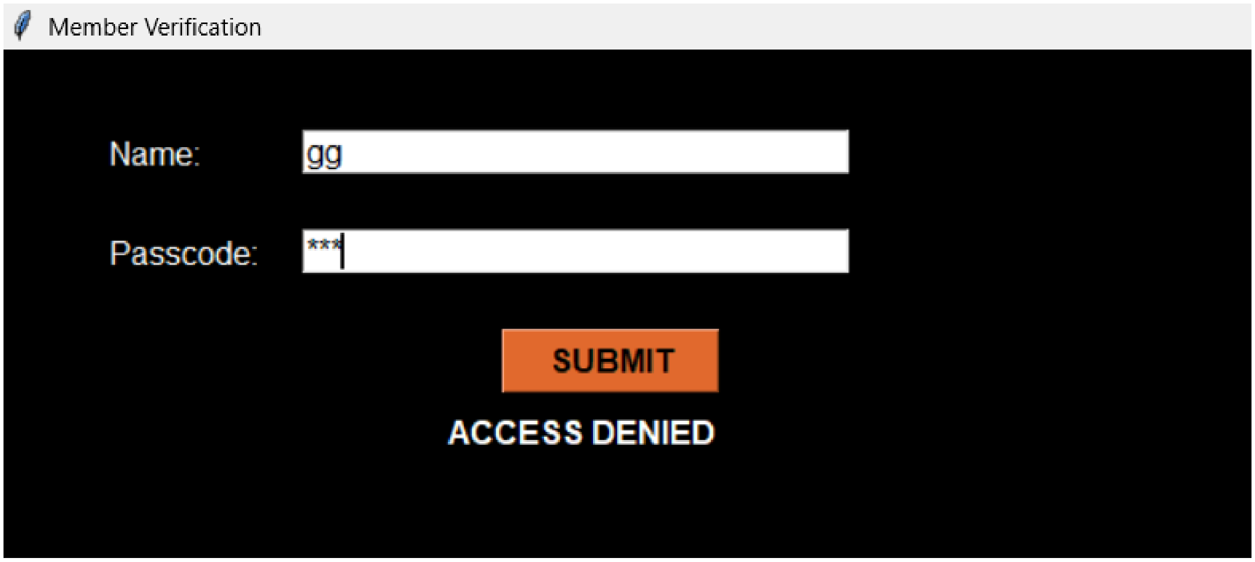
Clicking back returns to the previous window

If member:
Click the correct radiobutton and give submit

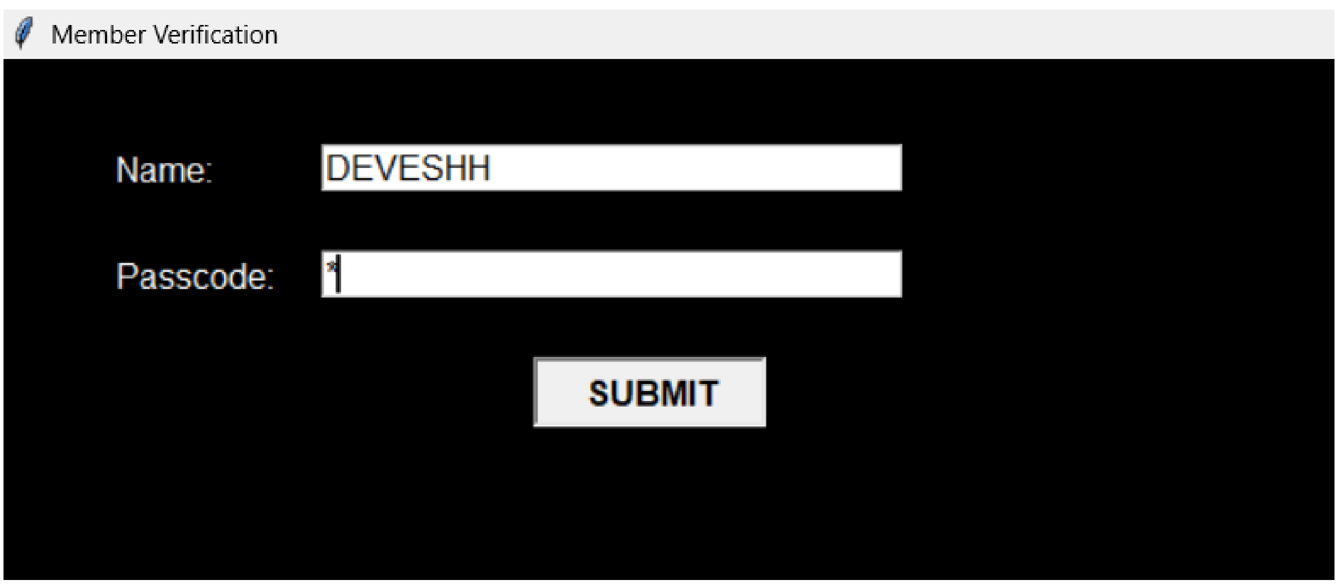


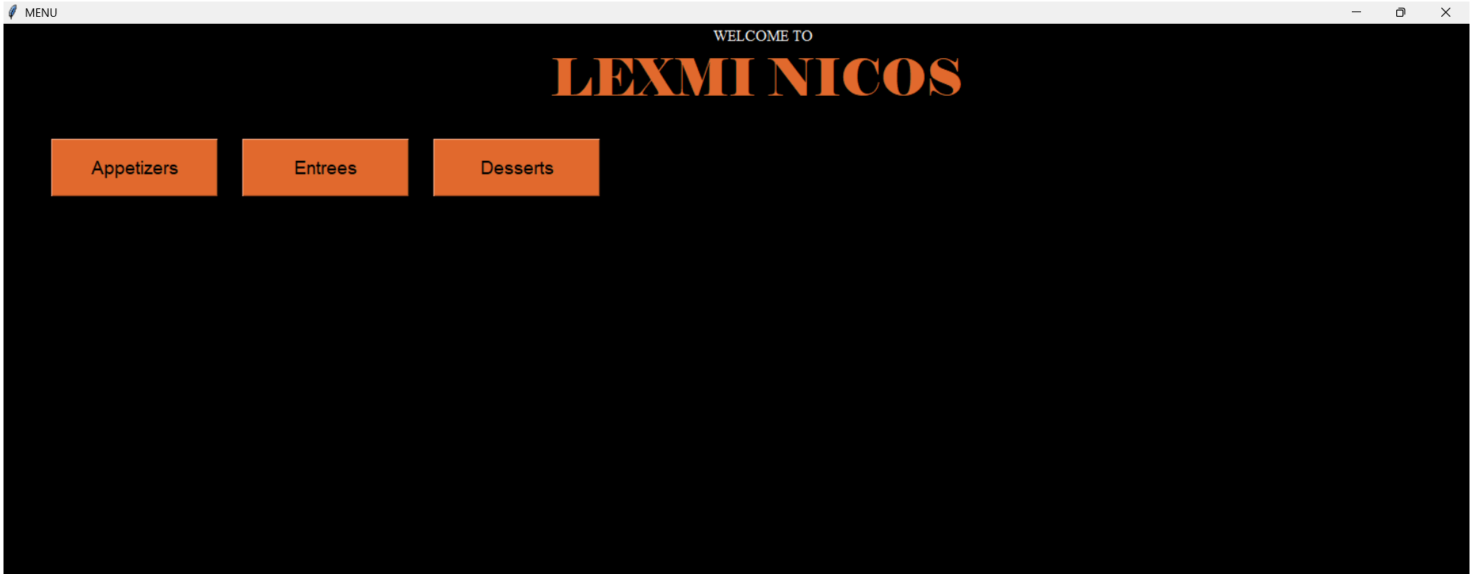


If incorrect details selected

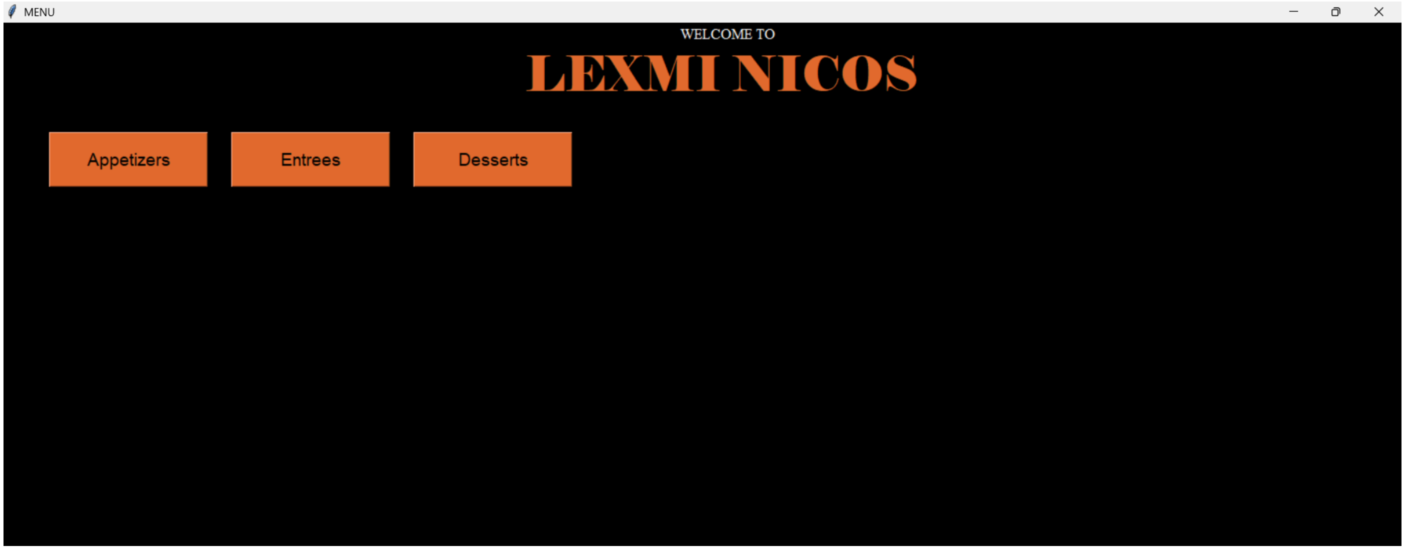


If correct details have been entered the following 2 images show what happens:

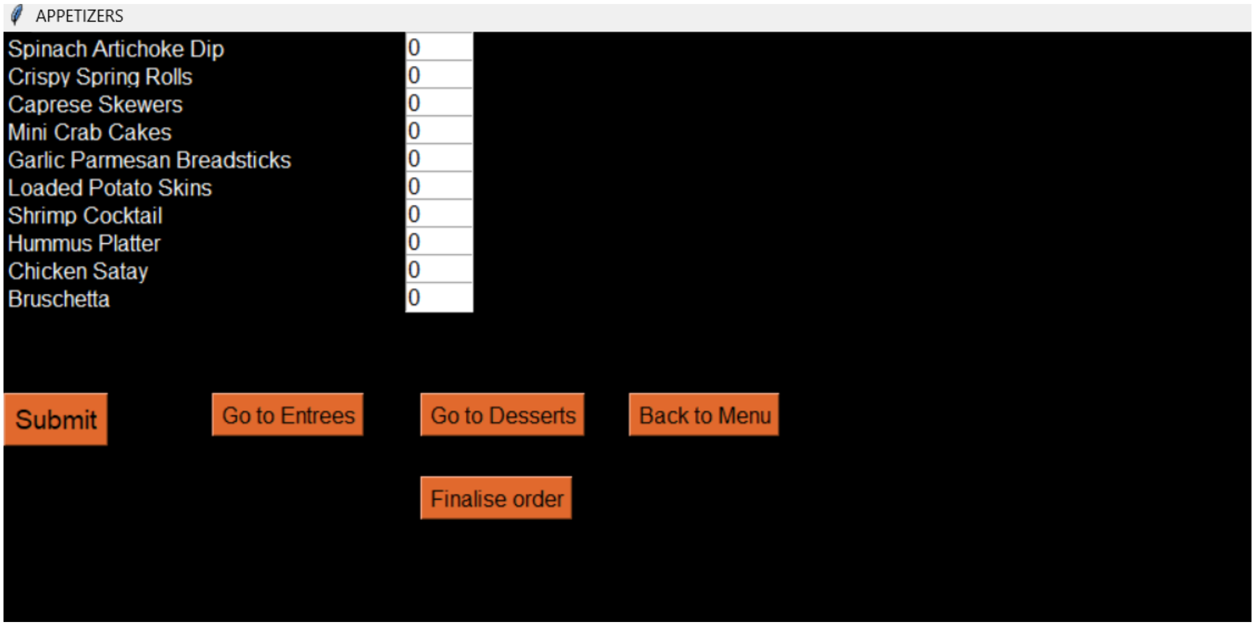


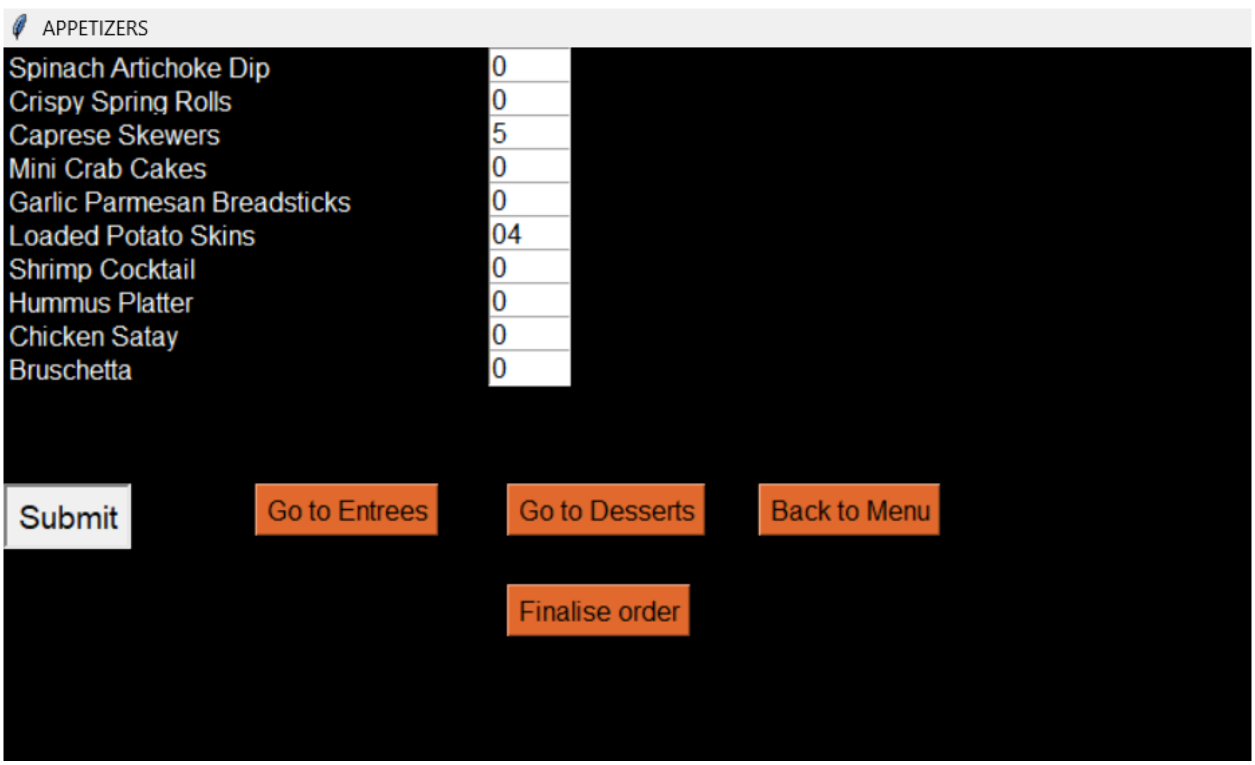


If directly not a member is selected the following image is shown:

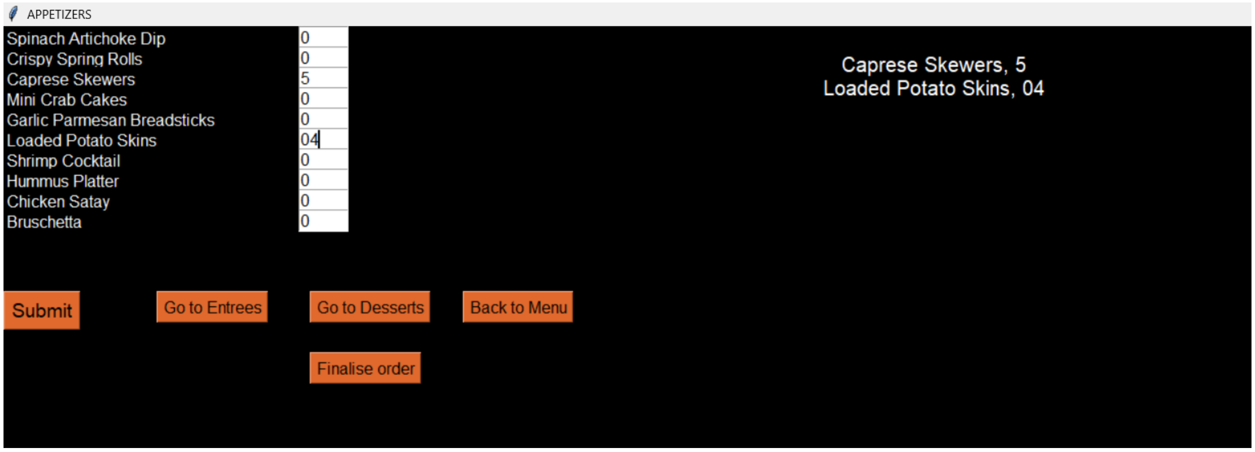


Appetizers is chosen



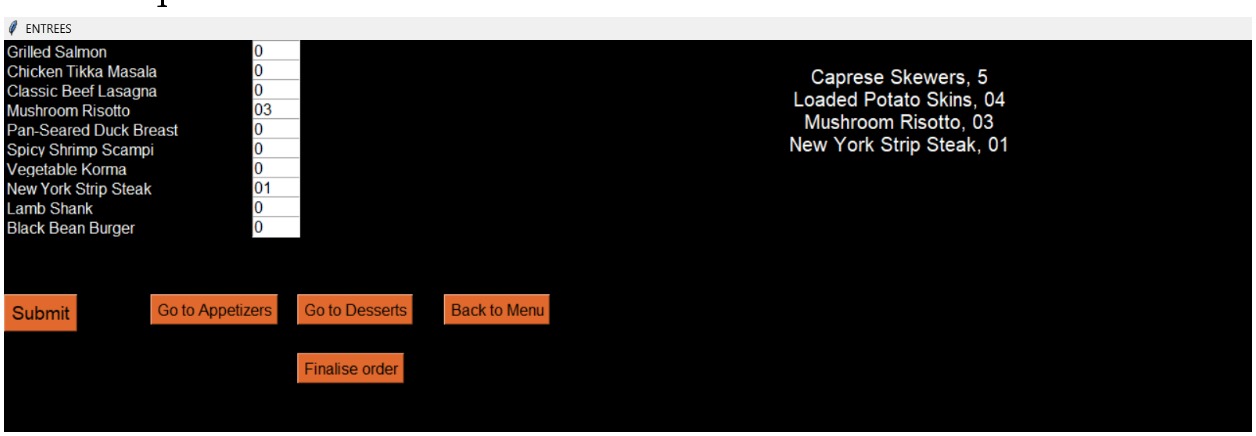


Submit is clicked then the following image illustrates what happens

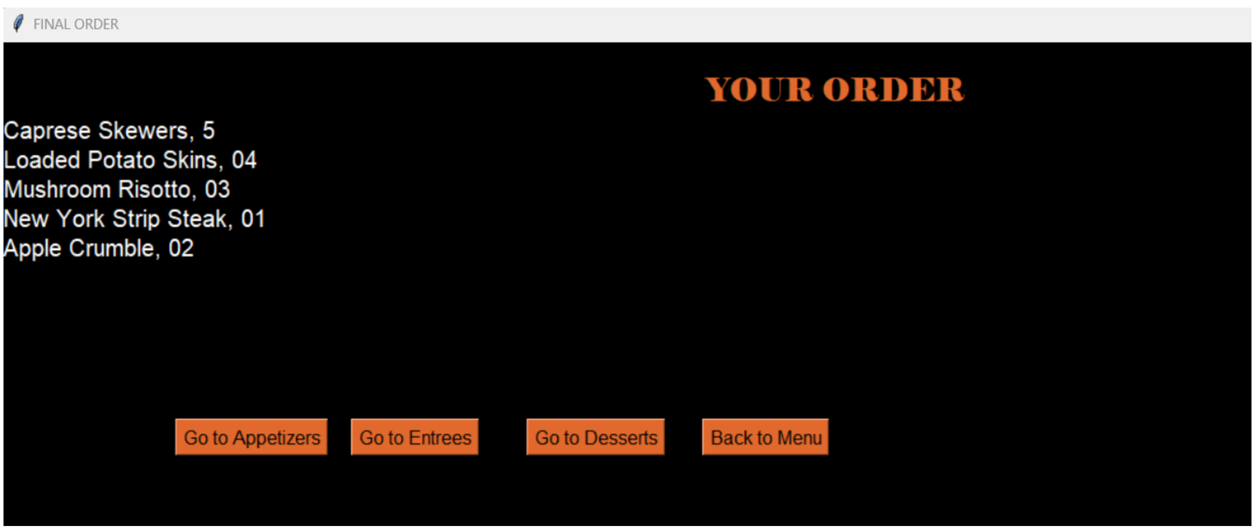


If entrees is clicked then the following image illustrates what happens

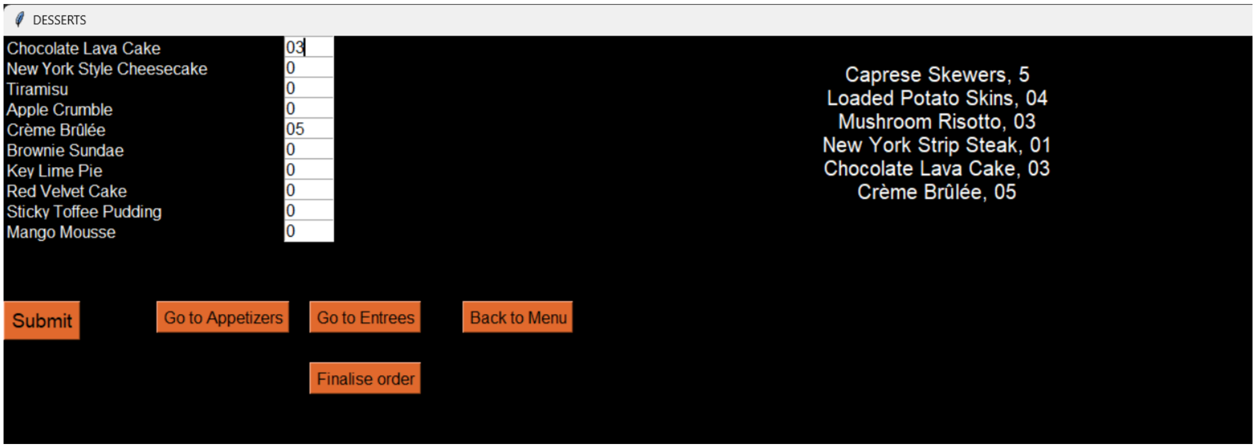
Wanted quantities are selected and submitted

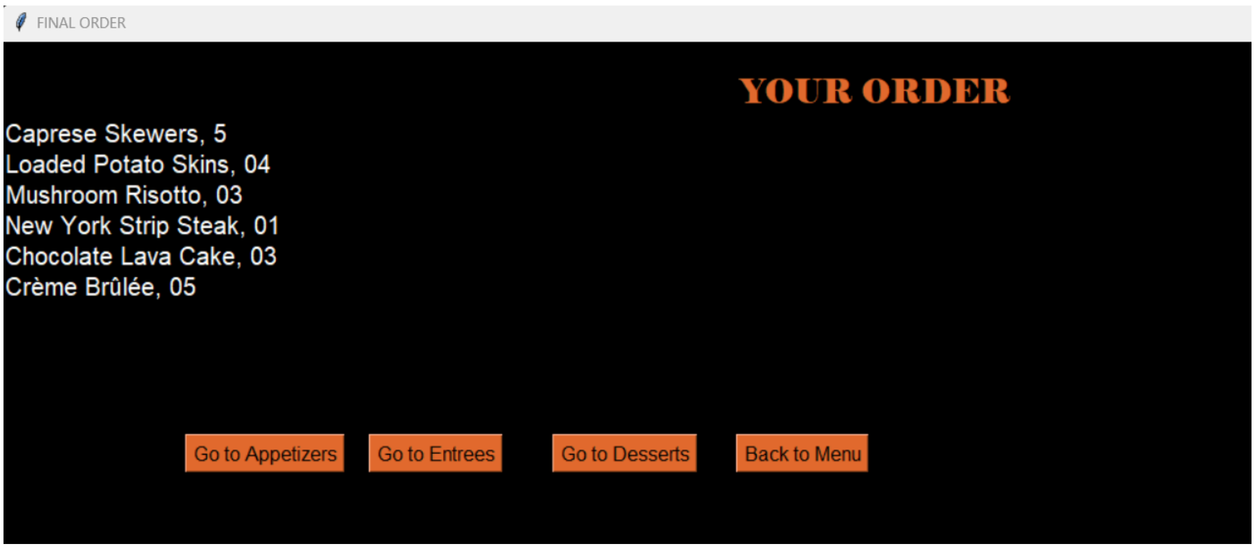


after all is selected finalize order is given



If in case there is any change to be made then u can click that section and change your order for example hear the desserts have been changed


DESSERTS
Chocolate Lava Cake
03
New York Style Cheesecake
0
Tiramisu
0
Apple Crumble
0
Crème Brûlée
05
Brownie Sundae
0
Key Lime Pie
0
Red Velvet Cake
0
Sticky Toffee Pudding
0
Mango Mousse
0
Caprese Skewers, 5
Loaded Potato Skins, 04
Mushroom Risotto, 03
New York Strip Steak, 01
Chocolate Lava Cake, 03
Crème Brûlée, 05
Submit
Go to Appetizers
Go to Entrees
Back to Menu
Finalise order


FINAL ORDER
YOUR ORDER
Caprese Skewers, 5
Loaded Potato Skins, 04
Mushroom Risotto, 03
New York Strip Steak, 01
Chocolate Lava Cake, 03
Crème Brûlée, 05
Go to Appetizers
Go to Entrees
Go to Desserts
Back to Menu

SCOPE FOR ENCHANCEMENTS:

1. Clicking of the staff menu can be linked up to a hotel management system
2. Order from users can be assigned a table number but for this project since it is used locally this has not been done
3. Automation of submit button and a feature to call the waiter incase of any problems

CONCLUSION

The *Lexmi Nicos Order Suite* stands as a refined integration of technology and service, combining the reliability of MySQL with the elegance of a graphical interface. Designed with clarity, precision, and poise, the system ensures seamless order placement while reflecting the sophistication of a well-orchestrated service experience. Beyond its immediate functionality, the project demonstrates how thoughtful design and disciplined engineering can elevate even routine tasks into an interface of distinction. In essence, this suite is not merely a program but a statement of how technology, when crafted with elegance, can serve with the grace of fine service.

BIBLIOGRAPHY:

Text References:
1. Sumita Arora, Computer Science with Python for Class XI (2023), Dhanpat Rai & Co. ISBN: 978-81-7700-236-0
2. Sumita Arora, Computer Science with Python for Class XI (2023), Dhanpat Rai & Co. ISBN: 978-81-7700-236-2

Web References:
1. https://youtu.be/i5Iv8KU9rLU?si=7zW1rRbAVa_-aaYJ
2. https://youtu.be/ScTgxrHqETI?si=o0YL4beJ7f9Td956
3. https://youtu.be/2MxzkMCXhCc?si=wK7jFgA1OYUwXcyG
4. https://youtu.be/nwht_3zUGI0?si=V_ZY3zKk3fo1feY0
5. https://youtu.be/oNAvlbL6utk?si=3CY7FQmAXKNUexcr
6. MySQL Documentation – *Oracle Corporation*. Official reference for database connectivity, queries, and schema design.
7. Python Documentation – *Python Software Foundation*. Official reference for Python programming language and libraries.
8. NCERT. (2025). *Computer Science – Class XII*. National Council of Educational Research and Training, New Delhi.